EMPLOYEE MANAGEMENT SYSTEM

Project Report

A React-Based HR Employee Directory Web Application

Prepared by:

Pranali Bagilgekar

GitHub: https://github.com/Pranali027-blip/employee-management-system

April 2026

1. Project Overview

The Employee Management System (EMS) is a React-based web application built to help HR teams and small businesses manage employee records efficiently. It provides a browser-based interface for adding, viewing, editing, and deleting employee profiles — without requiring a back-end or database setup.

The application is developed entirely on the front-end using React with hooks, making it lightweight, portable, and straightforward to set up. It follows modern UI/UX practices and is designed to be intuitive for non-technical users.

2. Objectives

- Build a fully functional employee directory using React
- Implement complete CRUD operations — Create, Read, Update, and Delete
- Provide real-time search and column-based sorting
- Display live statistics: total employees, unique roles, and average salary
- Demonstrate clean UI/UX design with dark mode support
- Practice form validation, React state management, and component-based design

3. Features

3.1 Core Functionality

Feature	Description
Add Employee	Form to create a new employee with name, role, and salary
Edit Employee	Modal popup to update existing employee details inline
Delete Employee	Remove an employee with instant visual confirmation
Real-Time Search	Filter employees by name or role as you type
Column Sorting	Click any header to sort ascending or descending
Live Statistics	Auto-updated headcount, unique roles, and average salary
Dark Mode	One-click toggle between light and dark themes
Toast Notifications	Brief on-screen confirmation for every action
Form Validation	Inline error messages for missing or invalid inputs

3.2 UI & Visual Highlights

- — Managers, Developers, Designers, Analysts, and HR each have distinct colors
- — Auto-generated from the employee's full name
- — Visual comparison of pay relative to the highest earner
- — Contextual prompts when no employees or search results exist
- — Clean layout suitable for non-technical users

4. Technology Stack

Technology	Version / Detail	Purpose
React	v18+	Core frontend UI framework
TypeScript	v5+	Type safety and better code quality
Vite	v5+	Build tool and dev server
useState Hook	React built-in	Component state management
useRef Hook	React built-in	Auto-incrementing ID counter
Custom CSS	Vanilla	Styling, theming, and dark mode
JavaScript ES6+	Modern JS	Application logic and data handling

5. Project Structure

The project follows the standard Vite + React folder structure:

```
vite-boot/
├── src/
│   ├── App.tsx          Main application component (all logic)
│   ├── App.css          Custom styles, badges, dark mode overrides
│   ├── main.tsx         React entry point
│   └── index.css        Global base styles
├── public/              Static assets (icons, favicons)
├── index.html           HTML shell / entry point
├── package.json         Project metadata and dependencies
├── vite.config.ts       Vite build configuration
└── tsconfig.json        TypeScript compiler configuration
```

6. Code Explanation

6.1 State Management

All application data is managed via React's useState hook. Separate state variables handle the employee list, form fields, validation errors, search query, sort key and direction, dark mode, edit modal, and toast messages. The useRef hook maintains an auto-incrementing employee ID counter that persists across renders without triggering re-renders.

6.2 CRUD Operations

Adding an employee first validates the form, then appends a new employee object to the array. Editing opens a pre-filled modal and replaces the matching record on save using .map(). Deleting filters out the employee by ID using .filter(). All three operations fire a toast notification immediately after completion.

6.3 Search and Sorting

Search filters the employee list in real time using JavaScript's .filter() and .toLowerCase() methods, matching against both name and role. Sorting uses .sort() with a comparator that handles strings and numbers, toggling direction when the same column is clicked twice.

6.4 Role Badges

The getBadgeClass() utility maps role keywords to CSS class names. It checks whether the role string contains keywords such as 'manager', 'developer', 'designer', 'analyst', or 'hr' and returns the appropriate CSS class for color-coded display in the table.

6.5 Dark Mode

Dark mode is implemented by toggling a 'dark' class on the document body. CSS class-based overrides in App.css handle all theme changes — no external library is required. The toggle button reflects the current mode with a sun or moon icon.

7. Screenshots

Figure 1 — Employee Directory (Main View with Stats)

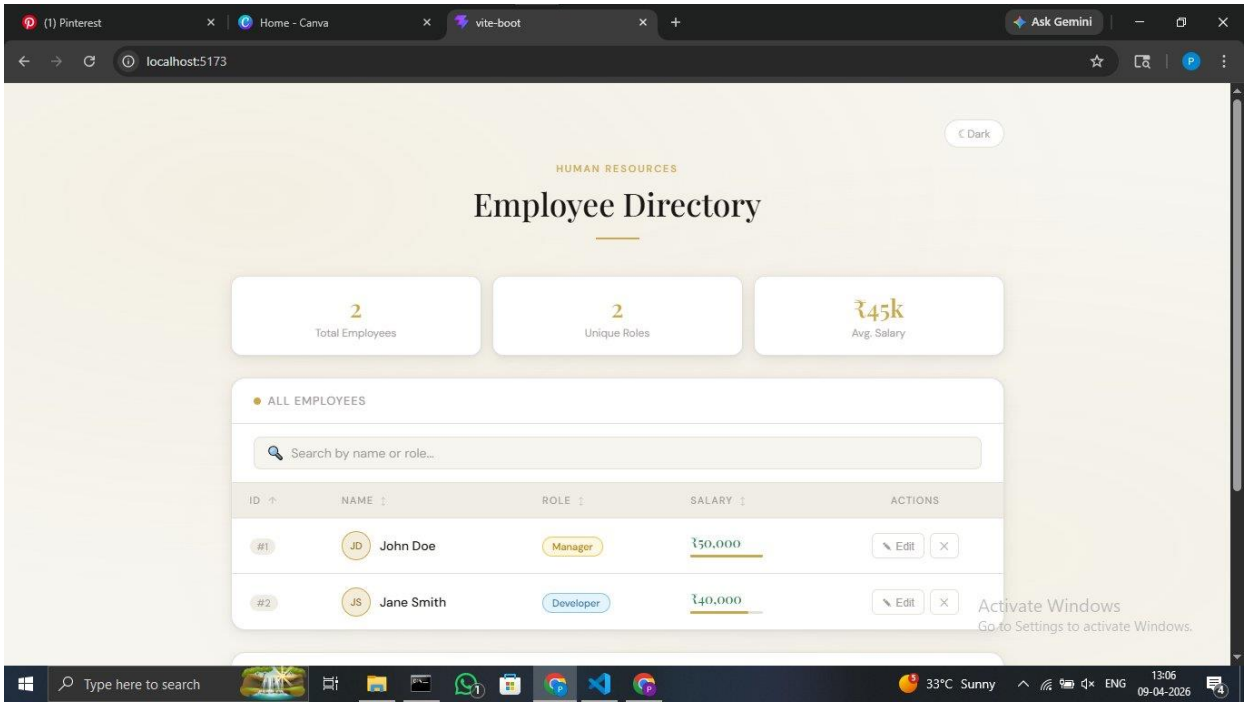
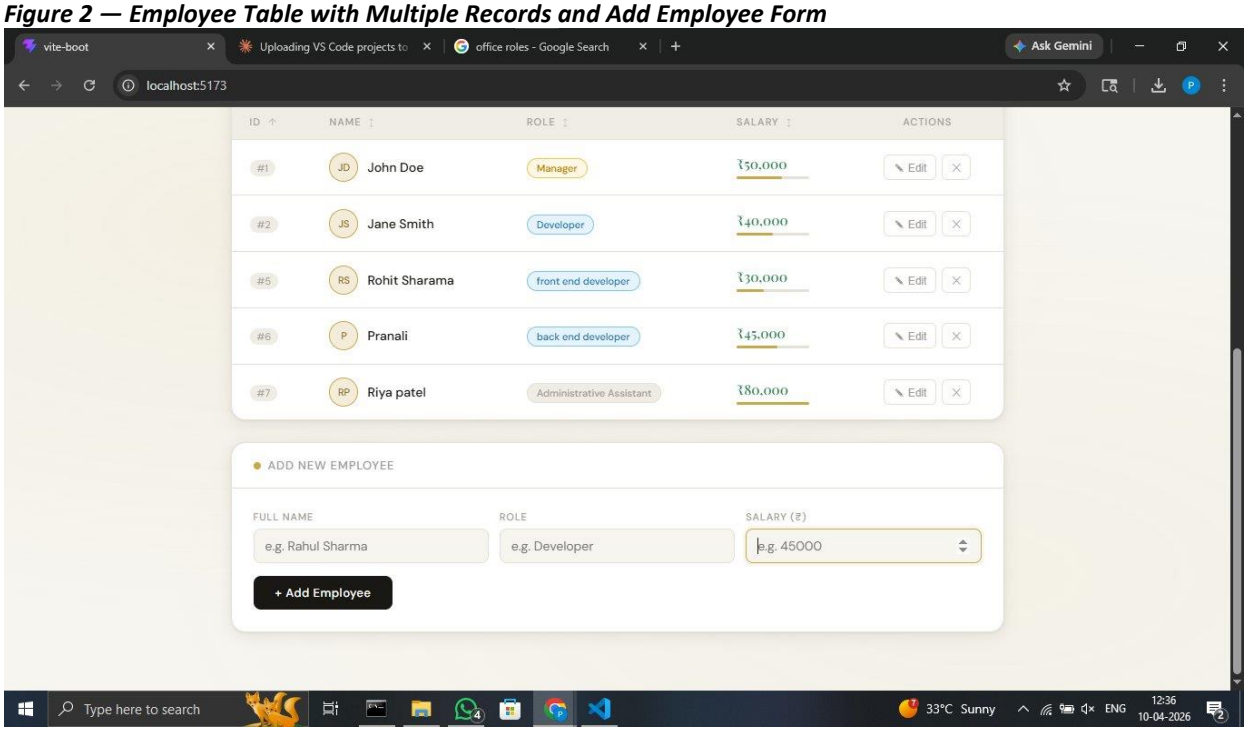


Figure 2 — Employee Table with Multiple Records and Add Employee Form



Screenshots are available in the GitHub repository.

8. How to Run the Project

8.1 Prerequisites

- Node.js v16 or above installed
- npm or yarn package manager
- Git installed and configured

8.2 Setup Instructions

Step	Command / Action
1. Clone the repository	git clone https://github.com/Pranali027-blip/employee-management-system.git
2. Enter the project folder	cd employee-management-system
3. Install dependencies	npm install
4. Start development server	npm run dev
5. Open in browser	http://localhost:5173

9. Limitations

- No back-end or database integration — data does not persist after page refresh
- No user authentication or authorization
- Currently optimized for desktop browsers; mobile responsiveness planned for future updates.
- No export functionality (e.g., CSV or PDF)
- Employee data is stored in React state only and resets on reload

10. Skills Demonstrated

This project highlights a range of technical and problem-solving skills relevant to modern front-end development:

- **React Development** — Utilized functional components, hooks such as useState and useRef, and component-based architecture
- **State Management** — Managed dynamic application data and UI updates efficiently using React hooks
- **TypeScript** — Applied static typing to improve code quality, maintainability, and error handling
- **CRUD Operations** — Implemented Create, Read, Update, and Delete functionalities for employee data management
- **User Interface (UI/UX) Design** — Designed an intuitive and user-friendly interface with attention to usability and accessibility
- **Frontend Architecture** — Structured the application using a modular and scalable approach
- **Problem Solving** — Handled real-time search, sorting logic, validation, and dynamic UI behavior

11. Conclusion

The Employee Management System demonstrates practical implementation of React concepts including CRUD operations, state management, form validation, real-time search, sorting, and UI theming. The project reflects an understanding of front-end architecture and modern web development practices.

It is structured as a front-end prototype demonstrating employee record management workflows. Future development could extend the application with back-end integration, persistent storage, user authentication, and mobile responsiveness.

12. Future Enhancements

The current version of the Employee Management System serves as a front-end prototype. The following enhancements can be implemented to extend its functionality and make it suitable for real-world deployment:

- **Backend Integration**
  Incorporate a backend using technologies such as Node.js or Firebase to handle server-side logic and API communication.
- **Database Implementation**
  Integrate a database like MongoDB or PostgreSQL to enable persistent storage of employee records.
- **User Authentication and Authorization**
  Implement secure login and role-based access control to restrict system usage to authorized users only.
- **Mobile Responsiveness**
  Optimize the user interface for mobile and tablet devices to ensure accessibility across all screen sizes.
- **Data Export Functionality**
  Add options to export employee data in formats such as CSV or PDF for reporting and record-keeping purposes.